```
            Student student = StudentsToStdWidth.toStudent(s);
            return student;
        }

        public static void update(RandomAccessFile f, int recNo, Student
    student) throws IOException {
            int records = (int) (f.length() / Student.RECSIZE);
            if (recNo < 0 || recNo >= records) {
                throw new RuntimeException("updating past of file end");
            }
            long pos = recNo * Student.RECSIZE;
            f.seek(pos);
            f.writeBytes(student.toFixedWidth());
        }

        public static void append(RandomAccessFile f, Student student) throws
    IOException {
            f.seek(f.length());
            f.writeBytes(student.toFixedWidth()+"\n");
        }

        public static void main(String[] args) throws FileNotFoundException,
    IOException {
            RandomAccessFile f = new
    RandomAccessFile("data"+File.separator+"students.dat", "rw");
            Student student = read(f, 3);
            System.out.println(student);
            student.setfName("Mike");
            student.setsName("Updater");
            update(f, 3, student);
            student=new Student("999", "Jane", "Appender",
    LocalDate.of(1999,Month.MARCH,20));
            student.setScores(new double[]{14,15,16,17,18,19});
            append(f,student);
            append(f,student);
            f.close();
        }
    }
```

Κώδικας 15.6 *Ανάγνωση και εγγραφή σε αρχεία με τυχαία προσπέλαση*

Η συνάρτηση read στον κώδικα 15.6 λαμβάνει ως παραμέτρους ένα αντικείμενο RandomAccessFile και έναν ακέραιο. Ο ακέραιος αναπαριστά τον αύξοντα αριθμό της εγγραφής μέσα στο αρχείο. Η συνάρτηση length της RandomAccessFile επιστρέφει το συνολικό μήκος του αρχείου σε ψηφιολέξεις. Το μήκος αυτό διαιρούμενο δια του μήκους της εγγραφής μας δίνει τον αριθμό των εγγραφών στο αρχείο. Έτσι, γίνεται έλεγχος μήπως ο αριθμός εγγραφής που ζητήθηκε από την read είναι εκτός των ορίων του αρχείου. Αν όχι, ο δείκτης τοποθετείται στη θέση recNo * Student.RECSIZE. Μετά την τοποθέτηση του δείκτη, διαβάζουμε μια εγγραφή και δημιουργούμε το αντικείμενο Student με την toStudent της StudentsToStdWidth.

Η συνάρτηση append μετακινεί τον δείκτη στο τέλος του αρχείου όπου προσθέτει μια νέα εγγραφή με τη βοήθεια της writeBytes και της toFixedWidth.

Η συνάρτηση update λαμβάνει ως παραμέτρους το αρχείο, τον αριθμό εγγραφής και ένα αντικείμενο Student. Αντικαθιστά τα στοιχεία της εγγραφής που βρίσκεται στη θέση recNo*Student.RECSIZE με τα στοιχεία του αντικειμένου Student. Προσέξτε πως στην writeBytes δεν προστίθεται χαρακτήρας newline όπως στην αντίστοιχη writeBytes της append, καθώς ο χαρακτήρας υπάρχει ήδη στο αρχείο.

15.5 Σειριοποίηση

Ας υποθέσουμε πως σε μια εφαρμογή-παιχνίδι θέλουμε να δώσουμε τη δυνατότητα στον χρήστη να διακόπτει το παιχνίδι και να το συνεχίζει ακριβώς από το σημείο στο οποίο διέκοψε. Στην περίπτωση αυτή θα μας ήταν

χρήσιμο να αποθηκεύσουμε σε αρχείο τα αντικείμενα της εφαρμογής όπως αυτά είχαν διαμορφωθεί στη μνήμη κατά τη στιγμή της διακοπής έτσι ώστε να μπορούμε να τα φορτώσουμε από το αρχείο στη μνήμη όποτε χρειαστεί. Η διαδικασία με την οποία επιτυγχάνουμε αποθήκευση και φόρτωση αντικειμένων μεταξύ μνήμης και αρχείου ονομάζεται σειριοποίηση (serialization).

Θα δούμε τη σειριοποίηση στην πράξη με ένα παράδειγμα το οποίο σώζει ένα ArrayList από αντικείμενα κλάσης που αναπαριστά αυτοκίνητα και στη συνέχεια το φορτώνει και το εμφανίζει. Θα χρειαστούμε καταρχάς μια κλάση που αναπαριστά τα αυτοκίνητα.

```
import java.io.Serializable;
import java.util.Objects;

public class Car implements Serializable {
    private String brand;
    private int maximumSpeed;

    public Car(String brand, int maximumSpeed) {
        this.brand = brand;
        this.maximumSpeed = maximumSpeed;
    }

    public int getMaximumSpeed() {
        return maximumSpeed;
    }

    public void setMaximumSpeed(int maximumSpeed) {
        this.maximumSpeed = maximumSpeed;
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    @Override
    public String toString() {
        return "Car{" + "brand=" + brand + ", maximumSpeed=" + maximumSpeed
+ '}';
    }

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 67 * hash + Objects.hashCode(this.brand);
        hash = 67 * hash + this.maximumSpeed;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
```

```
            }
            final Car other = (Car) obj;
            if (this.maximumSpeed != other.maximumSpeed) {
                return false;
            }
            return Objects.equals(this.brand, other.brand);
        }
    }
```

Κώδικας 15.7 *Η κλάση Car*

Η κλάση Car στον κώδικα 15.7 είναι μια απλή κλάση που περιλαμβάνει τις αναγκαίες συναρτήσεις. Εκείνο που έχει παραπάνω σε σχέση με τις κλάσεις που έχουμε υλοποιήσει μέχρι τώρα είναι πως υλοποιεί την προκαθορισμένη διεπαφή Serializable. Η διεπαφή Serializable δεν ορίζει καμία αφηρημένη μέθοδο. Πρόκειται για διεπαφή-δείκτη (marker interface). Η υλοποίησή της από οποιαδήποτε κλάση απαιτεί μόνο τη δήλωσή της στη λίστα των διεπαφών που υλοποιεί η κλάση. Ωστόσο είναι απαραίτητη ώστε να επιτρέπεται η σειριοποίηση μιας κλάσης.

Ας προχωρήσουμε τώρα στην εφαρμογή που αποθηκεύει το ArrayList από αντικείμενα Car.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

public class Serialization {

    static ArrayList<Car> createCars() {
        ArrayList<Car> store = new ArrayList<>();
        store.add(new Car("Toyota", 220));
        store.add(new Car("Ford", 230));
        store.add(new Car("Peugeot", 240));
        return store;
    }

    static void saveCars() throws IOException {
        ArrayList<Car> cars = createCars();
        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("data\\cars.dat"));
        out.writeObject(cars);
        out.close();
    }

    static ArrayList<Car> loadCars() throws IOException,
ClassNotFoundException {
        ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("data\\cars.dat"));
        ArrayList<Car> loaded = (ArrayList<Car>) in.readObject();//essential
unsafe operation
        in.close();
        return loaded;
    }

    public static void main(String[] args) throws IOException,
ClassNotFoundException {
        saveCars();
        ArrayList<Car> store = loadCars();
        System.out.println(store);
    }
}
```

Κώδικας 15.8 *Σειριοποίηση ενός ArrayList<Car>*

Στον κώδικα 15.8 η createCars δημιουργεί το ArrayList από αντικείμενα Car. Η saveCars ανοίγει ροή τύπου ObjectOutputStream που λειτουργεί ως περίβλημα σε FileOutputStream. Με την writeObject πραγματοποιείται η εγγραφή. Προσέξτε στην τεκμηρίωση της ArrayList πως η κλάση υλοποιεί τη διεπαφή Serializable, διαφορετικά η λειτουργία θα αποτύγχανε. Αντίστοιχα απλή είναι και η διαδικασία ανάγνωσης του αντικειμένου που γίνεται από την loadCars με χρήση ενός ObjectInputStream και ενός FileInputStream. Προσέξτε πως η readObject της ObjectInputStream επιστρέφει Object και άρα είναι απαραίτητη η μετατροπή του σε ArrayList<Car>.

15.6 Χρήσιμες λειτουργίες

Η κλάση File παρέχει μια σειρά από χρήσιμες λειτουργίες. Ένα αντικείμενο της κλάσης File αναπαριστά το όνομα της διαδρομής ενός αρχείου ή φακέλου [2]. Τα λειτουργικά συστήματα ονομάζουν τις διαδρομές των αρχείων με βάση τα ιδιαίτερα χαρακτηριστικά τους. Αυτά όμως διαφέρουν από λειτουργικό σε λειτουργικό. Καθώς η Java είναι διαπλατφορμική γλώσσα, χρειάζεται έναν τρόπο για να αναπαριστά τις ονομασίες των διαδρομών αρχείων και φακέλων που είναι ανεξάρτητος από το λειτουργικό σύστημα. Αυτόν τον τρόπο παρέχει η κλάση File.

Οι ονομασίες των διαδρομών αρχείων και φακέλων είναι δύο τύπων, οι σχετικές (relative) και οι πλήρεις (absolute). Η πλήρης διαδρομή επαρκεί για τον εντοπισμό του αρχείου που δηλώνει. Αντίθετα, μια σχετική διαδρομή πρέπει να συνδυαστεί με κάποια άλλη προκειμένου να εντοπιστεί το αρχείο που δηλώνει. Οι κλάσεις του πακέτου java.io επιλύουν τις σχετικές διαδρομές με βάση την ιδιότητα user.dir του συστήματος. Στην περίπτωσή μας, εφόσον δουλεύουμε στο NetBean, η διαδρομή που συνδυάζεται με την ιδιότητα user.dir αντιστοιχεί στον φάκελο του NetBeans project στο οποίο εργαζόμαστε.

Ας δούμε τώρα μια σειρά από χρήσιμες συναρτήσεις:

```
public static String getProperty(String key)
```

Η συνάρτηση getProperty της κλάσης System όταν κληθεί με παράμετρο “user.dir” επιστρέφει τον τρέχοντα φάκελο εργασίας.
Οι υπόλοιπες συναρτήσεις ανήκουν στην κλάση File.

```
boolean canExecute()
```

Ελέγχει αν η εφαρμογή μπορεί να τρέξει αυτό το αρχείο, δηλαδή το αρχείο που αναπαρίσταται από το αντικείμενο File που καλεί την canExecute.

```
boolean canRead()
```

Ελέγχει αν η εφαρμογή μπορεί να διαβάσει αυτό το αρχείο.

```
boolean canWrite()
```

Ελέγχει αν η εφαρμογή μπορεί να γράψει σε αυτό το αρχείο

```
boolean createNewFile()
```

Δημιουργεί ένα αρχείο με αυτό το όνομα εφόσον δεν υπάρχει ήδη.

```
boolean delete()
```

Διαγράφει το αρχείο ή τον φάκελο με αυτό το όνομα.

```
String getAbsolutePath()
```

Επιστρέφει την πλήρη διαδρομή αυτού του αρχείου.

```
String getName()
```

Επιστρέφει το όνομα αυτού του αρχείου.

```
boolean isDirectory()
```

Ελέγχει αν αυτό το αρχείο είναι φάκελος.

```
boolean isHidden()
```
Ελέγχει αν αυτό το αρχείο είναι κρυφό.

```
long lastModified()
```
Επιστρέφει τον χρόνο σε milisecond από την τελευταία τροποποίηση αυτού του αρχείου.

```
long length()
```
Επιστρέφει το μέγεθος σε bytes αυτού του αρχείου.

```
File[] listFiles()
```
Επιστρέφει έναν πίνακα από αρχεία που περιέχονται σε αυτόν τον φάκελο.

```
boolean mkdir()
```
Δημιουργεί έναν φάκελο με αυτό το όνομα.

15.7 Λυμένες Ασκήσεις

15.7.1 Περιοδικός Πίνακας

Σε ένα οριοθετημένο αρχείο κειμένου περιέχονται **όλα** τα χημικά στοιχεία του περιοδικού πίνακα με τα βασικά τους χαρακτηριστικά όπως φαίνεται στο ακόλουθο παράδειγμα:

```
H 1 1 s 1
Li 1 2 s 3
Na 1 3 s 11
Be 2 2 s 4
Mg 2 3 s 12
Sc 3 4 d 21
Y 3 5 d 39
He 18 1 p 2
La 3 6 f 57
```

Η γραμμογράφηση του αρχείου έχει ως εξής:

```
Σύμβολο Ομάδα Περίοδος Τομέας Ατομικός_Αριθμός
```

Προσθέστε στην κλάση TableOfChemichalElements συνάρτηση

```
public static TableOfChemichalElements getPeriodTable()
```

που επιστρέφει ένα αντικείμενο TableOfChemichalElements που περιλαμβάνει τα στοιχεία του αρχείου και επομένως αναπαριστά τον περιοδικό πίνακα. Φροντίστε ώστε η getPeriodTable να διαβάζει το αρχείο μόνο μία φορά κατά την τρέχουσα εκτέλεση του προγράμματος. Μετά την ανάγνωση να διατηρεί την επιστρεφόμενη τιμή στη μνήμη από όπου την επιστρέφει με κάθε κλήση της.

Λύση
Καταρχάς θα πρέπει να προσθέσουμε δύο στατικά μέλη στην κλάση.

```
private final static File F = new File("data" + File.separator
+ "ChemicalElements.dat");
private static TableOfChemichalElements periodTable = null;
```

Το F αφορά το αρχείο το οποίο περιέχει τα χημικά στοιχεία. Η getPeriodTable δεν θα φορτώνει δεδομένα από ένα οποιοδήποτε αρχείο αλλά από ένα συγκεκριμένο που περιέχει το σύνολο των χημικών στοιχείων. Με αυτήν την έννοια, το αρχείο δεν σχετίζεται με συγκεκριμένο στιγμιότυπο της TableOfChemichalElements, οπότε το F είναι static. Επίσης static είναι το periodTable που είναι ο πίνακας που επιστρέφει η getPeriodTable. Το periodTable αρχικοποιείται στην τιμή null. Μετά την πρώτη εκτέλεση της getPeriodTable αποτελεί αναφορά στο στιγμιότυπο TableOfChemichalElements που επιστρέφει η getPeriodTable.
Στη συνέχεια, αναπτύσσουμε τρεις εναλλακτικές λύσεις και συζητάμε τα πλεονεκτήματα και μειονεκτήματά τους.

```
     public static TableOfChemichalElements getPeriodTable() throws
FileNotFoundException {
        if (periodTable == null) {
            periodTable = new TableOfChemichalElements();
            Scanner scanner = new Scanner(F);
            while (scanner.hasNextLine()) {
                String line = scanner.nextLine();
                String[] elements = line.split(" ");
                ChemicalElement cE = new
ChemicalElement(Integer.parseInt(elements[1]), Integer.parseInt(elements[2]),
elements[3].charAt(0), Integer.parseInt(elements[4]), elements[0]);
                periodTable.add(cE);
            }
            scanner.close();
        }
        return periodTable;
    }
```

Κώδικας 15.9 *Η συνάρτηση getPeriodTable*

Στον κώδικα 15.9 η συνάρτηση getPeriodTable ελέγχει τη στατική μεταβλητή periodTable και αν την βρει null φορτώνει το αρχείο με τα χημικά στοιχεία στη μνήμη. Στη συνέχεια επιστρέφει την periodTable. Πράγματι με αυτήν την προσέγγιση θα επιστρέψει στον χρήστη της TableOfChemichalElements έναν πίνακα με χημικά στοιχεία. Έχει ενδιαφέρον όμως τι μπορεί να συμβεί από κει και πέρα.

```
     public static void main(String[] args) throws IOException {
        TableOfChemichalElements pT=TableOfChemichalElements.getPeriodTable();
        ChemicalElement li = new ChemicalElement(1, 2, 's', 3, "Pi");
        pT.add(li);
        System.out.println(TableOfChemichalElements.getPeriodTable());

    }
```

Κώδικας 15.10 *Αλλοίωση του πίνακα περιοδικών στοιχείων από την εφαρμογή*

Η getPeriodTable επιστρέφει την αναφορά στον TableOfChemicalElements.periodTable. Ο κώδικας 15.10 κρατάει αυτήν την αναφορά στην τοπική μεταβλητή pT. Επομένως, οι αναφορές TableOfChemicalElements.periodTable και pT βλέπουν στον ίδιο πίνακα στη μνήμη. Από τη στιγμή που θα γίνει μια μεταβολή στοιχείων του πίνακα από την εφαρμογή, π.χ. στον κώδικα 15.10 προστίθεται ένα στοιχείο, κάθε κλήση της getPeriodTable θα επιστρέφει όχι τον αρχικό πίνακα, αυτόν που φορτώθηκε από το αρχείο, αλλά τον πίνακα όπως μεταβλήθηκε από την εφαρμογή. Πρόκειται όμως για τον περιοδικό πίνακα των χημικών στοιχείων που είναι συγκεκριμένος και θα πρέπει πάντα να επιστρέφεται αμετάβλητος. Επομένως, η προσέγγιση αυτή παρουσιάζει σοβαρή αδυναμία.

Η λύση στο πρόβλημα αυτό είναι η getPeriodTable να δημιουργεί έναν νέο πίνακα, να τον ενημερώνει κατάλληλα και να τον επιστρέφει στον χρήστη της. Έτσι κάθε μεταβολή του πίνακα από την εφαρμογή δεν θα επηρεάζει τον πίνακα που θα επιστρέφει η getPeriodTable. Για να το πετύχουμε αυτό, θα πρέπει καταρχάς να προσθέσουμε δύο νέους δημιουργούς στην TableOfChemicalElements.

```
    public TableOfChemichalElements() {}
```

```
public TableOfChemichalElements(TableOfChemichalElements source) {
    Iterator<ChemicalElement> it = source.store.iterator();
    while (it.hasNext()) {
        store.add(it.next());
    }
}
```

Κώδικας 15.11 *Δύο δημιουργοί της κλάσης TableOfChemicalElements.*

Ο δημιουργός που λαμβάνει ως παράμετρο TableOfChemicalElements παράγει ένα αντίγραφο της παραμέτρου του. Προσέξτε πως τα χημικά στοιχεία στο αντίγραφο είναι τα ίδια με τα χημικά στοιχεία της παραμέτρου και όχι αντίγραφά τους. Αυτό είναι σωστό καθώς τα χημικά στοιχεία δεν διαθέτουν ρυθμιστές και επιπλέον όλες οι μεταβλητές στιγμιότυπου είναι σταθερές. Επομένως, ένα χημικό στοιχείο από τη στιγμή που θα δημιουργηθεί δεν μπορεί να μεταβληθεί και άρα δεν υπάρχει λόγος να παράγουμε αντίγραφο.

Ο δημιουργός χωρίς παραμέτρους είναι απαραίτητος για να τρέχουν οι κώδικες που μέχρι τώρα χρησιμοποιούσαν τον εξ ορισμού δημιουργό.

Έχοντας τους δύο δημιουργούς στη διάθεσή μας αρκεί να μετατρέψουμε την τελευταία γραμμή της getPeriodTable ώστε, αντί να επιστρέφει τη στατική periodTable, να επιστρέφει ένα αντίγραφό της όπως δείχνει ο κώδικας που ακολουθεί:

```
return new TableOfChemichalElements(periodTable);
```

Μία εναλακτική προσέγγιση βασίζεται στην αξιοποίηση της διεπαφής Cloneable. Όσες κλάσεις υλοποιούν τη διεπαφή Cloneable τους επιτρέπεται να καλέσουν τη συνάρτηση clone() που κληρονομούν από την Object. Εδώ το σχήμα φαίνεται λίγο περίεργο καταρχάς, αλλά έχει την εξήγησή του. Η συνάρτηση clone() που επιστρέφει ένα αντίγραφο του αντικειμένου που την καλεί τοποθετήθηκε στην κλάση Object σε αρχική έκδοση της Java. Επομένως, όλες οι κλάσεις την κληρονομούν. Σύντομα όμως έγινε αντιληπτό πως κάποιες κλάσεις δεν θα πρέπει να δίνουν τη δυνατότητα παραγωγής αντιγράφων των αντικειμένων τους. Προκειμένου να μην βγει η clone από την Object ώστε να διατηρηθεί η συμβατότητα με προηγούμενες εκδόσεις, οι σχεδιαστές της Java αποφάσισαν να ορίσουν τη διεπαφή Cloneable και να επιτρέψουν τη χρήση της clone στις κλάσεις που υλοποιούν την Cloneable. Επομένως, ο λόγος της περίεργης απαίτησης υλοποίησης μιας διεπαφής προκειμένου να υλοποιηθεί μια μέθοδος που κληρονομείται κρύβεται πίσω από μια αρχικά λανθασμένη σχεδιαστική επιλογή και μια εκ των υστέρων ευρετική λύση. Ας σημειωθεί πως οι κλάσεις που επιχειρούν να καλέσουν την clone χωρίς να έχουν υλοποιήσει τη διεπαφή clone παράγουν εξαίρεση τύπου CloneNotSupportedException.

Για την υλοποίηση αυτής της εναλακτικής θα πρέπει καταρχάς να δηλώσουμε στην επικεφαλίδα της TableOfChemicalElements ότι υλοποιεί τη διεπαφή Cloneable, όπως φαίνεται αμέσως παρακάτω:

```
public class TableOfChemichalElements implements Cloneable {…}
```

Στη συνέχεια πρέπει να επανορίσουμε τη συνάρτηση clone στην TableOfChemichalElements.

```
@Override
public Object clone() throws CloneNotSupportedException {
    TableOfChemichalElements rVal=new TableOfChemichalElements();
    Iterator<ChemicalElement> it = store.iterator();
    while (it.hasNext()) {
        rVal.add(it.next());
    }
    return rVal;
}
```

Κώδικας 15.12 *Η συνάρτηση clone της κλάσης TableOfChemicalElements.*

Προσέξτε πως η clone παράγει CloneNotSupportedException και επιστρέφει Object. Στη συνέχεια θα τροποποιήσουμε την getPeriodTable ως εξής:

```
    public static TableOfChemichalElements getPeriodTable() throws
FileNotFoundException, CloneNotSupportedException {
        if (periodTable == null) {
            periodTable = new TableOfChemichalElements();
            Scanner scanner = new Scanner(F);
            while (scanner.hasNextLine()) {
                String line = scanner.nextLine();
                String[] elements = line.split(" ");
                ChemicalElement cE = new
ChemicalElement(Integer.parseInt(elements[1]), Integer.parseInt(elements[2]),
elements[3].charAt(0),
                        Integer.parseInt(elements[4]), elements[0]);
                periodTable.add(cE);
            }
            scanner.close();
        }
        return (TableOfChemichalElements)periodTable.clone();
    }
```

Κώδικας 15.13 *Η συνάρτηση getPeriodTable βασισμένη στην clone.*

Στον κώδικα 15.13 η επιστροφή της συνάρτησης βασίζεται στην clone. Καθώς η clone επιστρέφει αντικείμενο τύπου Object, πρέπει υποχρεωτικά να προβούμε σε μετατροπή τύπου. Επιπλέον, στη συνάρτηση έχει δηλωθεί η πιθανή παραγωγή της CloneNotSupportedException.

Ανακεφαλαιώνοντας επισημαίνουμε πως η πρώτη λύση παρουσιάζει σοβαρή αδυναμία που μπορεί να οδηγήσει σε λάθη τον κώδικα μιας εφαρμογής. Αντίθετα, η δεύτερη και τρίτη λύση προστατεύει τον περιοδικό πίνακα. Οι χρήστες της κλάσης μπορούν να κάνουν μεταβολές στον πίνακα που θα λάβουν από την getPeriodTable αλλά η τιμή επιστροφής της μένει αμετάβλητη. Στην πράξη η δεύτερη και η τρίτη λύση είναι σε γενικές γραμμές ισοδύναμες.

15.7.2 Κατάλογοι αρχείων

Να υλοποιηθεί συνάρτηση public static void dirRecurse(File root) που λαμβάνει ως παράμετρο και εμφανίζει τα περιεχόμενα του root. Αν το root αντιστοιχεί σε κατάλογο αρχείων, εμφανίζει στοιχεία για κάθε αρχείο του καταλόγου και το ίδιο κάνει για κάθε υποκατάλογο του root. Αν το root αντιστοιχεί σε απλό αρχείο, εμφανίζει πληροφορίες για το αρχείο. Για κάθε κατάλογο από το root και κάτω θα πρέπει να εμφανίζεται πρώτα η ονομασία του και να ακολουθεί λίστα με πληροφορίες για κάθε αρχείο του καταλόγου σύμφωνα με το υπόδειγμα που ακολουθεί:

```
   Directory: G:\My Drive\Code\Kallipos\book\data
-rw     414 B      students.dat
-rw     220 B      studentsd.dat
-rw     75 KB      sunset.jpg
-rw     75 KB      sunsetcopy.jpg
-rw     75 KB      sunsetcopyf.jpg
-rw     199 B      cars.dat
-rw     122 B      doubledata.dat
-rw     103 B      ChemicalElements.dat

   Directory: G:\My Drive\Code\Kallipos\book\data\test
-rw      72 B      Calendar.txt
-rw     16 KB     Letter.docx
hrw       0 B      demohidden.txt
```

Στο υπόδειγμα αυτό η dirRecurse κλήθηκε με παράμετρο το αντικείμενο File που αντιστοιχεί στον φάκελο G:\My Drive\Code\Kallipos\book\data. Ο φάκελος περιέχει τον υποφάκελο test. Οι πληροφορίες που αφορούν κάθε αρχείο έχουν ως εξής: Οι πρώτοι τρεις χαρακτήρες δηλώνουν κατά σειρά αν το αρχείο είναι κρυφό

(hidden), αν επιτρέπεται η ανάγνωση και η εγγραφή. Στη συνέχεια ακολουθεί το μέγεθος του αρχείου και τέλος το όνομά του.

Λύση
Καταρχάς θα συγκεντρώσουμε όλες τις συναρτήσεις που θα μας χρειαστούν για να δομήσουμε την dirRecurse σε μία κλάση την οποία ονομάζουμε Directory.

```
import java.io.File;
import java.util.Locale;

public class Directory {

    public static String convertBytes(long bytes) {
        String rVal;
        long kB = bytes / 1024;
        long mB = bytes / (long) Math.pow(1024, 2);
        long gB = bytes / (long) Math.pow(1024, 3);
        if (gB > 0) {
            rVal = gB + " GB";
        } else if (mB > 0) {
            rVal = mB + " MB";
        } else if (kB > 0) {
            rVal = kB + " KB";
        } else {
            rVal = bytes + " B";
        }
        return rVal;
    }

    public static void dirRecurse(File root) {
        if (root.isDirectory()) {
            System.out.println("   Directory: " + root.getAbsolutePath());
            showFiles(root.listFiles());
        } else {
            System.out.println((root.isHidden() ? "h" : "-")
                    + (root.canRead() ? "r" : "-")
                    + (root.canWrite() ? "w" : "-")
                    + String.format(Locale.US, "%10s",
convertBytes(root.length()))
                    + "      " + root.getName());
        }
    }

    public static void showFiles(File[] files) {
        for (File file : files) {
            if (file.isDirectory()) {
                System.out.println("");
                System.out.println("   Directory: " +
file.getAbsolutePath());
                showFiles(file.listFiles()); // Calls same method again.
            } else {
                System.out.println((file.isHidden() ? "h" : "-")
                        + (file.canRead() ? "r" : "-")
                        + (file.canWrite() ? "w" : "-")
                        + String.format(Locale.US, "%10s",
convertBytes(file.length()))
                        + "      " + file.getName());
            }
        }
    }
```

```
        public static void main(String[] args) {
            File root = new File("data");
            dirRecurse(root);
        }
    }
```

Κώδικας 15.14 *Η κλάση Directory.*

Η πιο κρίσιμη ίσως συνάρτηση της Directory στον κώδικα 15.14 είναι η showFiles. Η showFiles λαμβάνει ως παράμετρο έναν πίνακα από αντικείμενα Files και για κάθε ένα από αυτά ελέγχει και αν είναι φάκελος, τυπώνει τη διαδρομή του και προχωράει σε αναδρομική κλήση με τη λίστα που αναπαριστά τα αρχεία του τρέχοντος φακέλου. Αν όμως πρόκειται για απλό αρχείο, τότε απλώς εμφανίζει τις απαιτούμενες πληροφορίες. Οι πληροφορίες παράγονται με αξιοποίηση των κατάλληλων συναρτήσεων της κλάσης File. Ειδικά για το μέγεθος του αρχείου χρησιμοποιείται και η συνάρτηση convertBytes. Καθώς η συνάρτηση length επιστρέφει το μέγεθος του αρχείου σε bytes, υπάρχει η ανάγκη να μετατραπεί σε KB, MB ή GB ανάλογα με τον αριθμό των bytes. Τη μετατροπή αυτή αναλαμβάνει η convertBytes, η οποία διαιρεί τον αριθμό των bytes με το 1024, το 1024^2 και το 1024^3 ώστε να βρει την κατάλληλη έκφραση για το μέγεθος του αρχείου. Αν η διαίρεση δώσει θετικό αριθμό GB, τότε το μέγεθος θα εκφραστεί σε GB. Αν όμως δεν προκύψει από τη διαίρεση θετικός αριθμός GB, περίπτωση κατά την οποία το μέγεθος του αρχείου είναι μικρότερο από 1 GB, τότε εξετάζεται η περίπτωση που το μέγεθος του αρχείου είναι μεγαλύτερο από 1 MB ή 1KB.

Τέλος η dirRecurse αποτελεί περίβλημα της showFiles. Η dirRecurse λαμβάνει ως παράμετρο ένα αντικείμενο File. Αν είναι φάκελος, τυπώνει την ονομασία του και καλεί την showFiles με τη λίστα αρχείων του φακέλου. Αν όμως η παράμετρος της dirRecurse αντιστοιχεί σε αρχείο, εμφανίζει απλώς τα στοιχεία του.

15.8 Ασκήσεις προς Λύση

15.8.1 Quiz 2η έκδοση

Να τροποποιηθεί η κλάση Quiz έτσι ώστε οι ερωτήσεις να φορτώνονται από αρχείο. Η κλάση θα πρέπει να συνεχίσει να υποστηρίζει ερωτήσεις σωστού λάθους και πολλαπλής επιλογής. Το αρχείο στο οποίο βρίσκονται οι ερωτήσεις να δίνεται ως παράμετρος της main.

15.8.2 Grep

Οι εντολές grep (global regular expression print) ψάχνουν σε μία ομάδα αρχείων για να βρουν μια συμβολοσειρά. Σε αυτήν την άσκηση θα πρέπει να αναπτύξουμε μια συνάρτηση public static void grep(File file, String searchItem) που λαμβάνει δύο παραμέτρους. Αν η πρώτη παράμετρος αντιστοιχεί σε απλό αρχείο, ψάχνει στο αρχείο να βρει τη συμβολοσειρά searchItem και εμφανίζει κάθε γραμμή του αρχείου που περιέχει την searchItem μαζί με τον αριθμό που έχει η γραμμή μέσα στο αρχείο. Αν η πρώτη παράμετρος είναι φάκελος, εφαρμόζει την αναζήτηση που περιγράφηκε για κάθε αρχείο του φακέλου και για τα αρχεία όλων των υποφακέλων του. Ακολουθήστε το παρακάτω υπόδειγμα εξόδου της grep:

students.dat

```
(1)   113            Jim           Blue10.011.012.013.014.015.02005-03-01
(5)   999            Jim       Appender14.015.016.017.018.019.01999-03-20
```

studentsd.dat

```
(1) 113 Jim Blue 10.0 11.0 12.0 13.0 14.0 15.0 2005-04-01
```

Ο αριθμός μέσα στις παρενθέσεις αναπαριστά τον αριθμό γραμμής του αρχείου όπου βρέθηκε το searchItem. Στο συγκεκριμένο παράδειγμα το searchItem είναι η σειρά Jim.

15.8.3 CopyFile

Αναπτύξτε java εφαρμογή που λαμβάνει ως παραμέτρους δύο συμβολοσειρές source και target και αντιγράφει το αρχείο με διαδρομή το source στο αρχείο target. Οι παράμετροι source και target θα πρέπει να περνούν στην main. Ελέγξτε αν η εφαρμογή σας λειτουργεί σωστά τόσο με αρχεία κειμένου όσο και με αρχεία εικόνας.

Βιβλιογραφία

[1] C. Horstmann, Η γλώσσα προγραμματισμού Java, Αναλυτική Προσέγγιση. Broken Hill Publishers, 2021.

[2] J. Farrell, Java, Εκμάθηση με πρακτικά παραδείγματα. Κριτική, 2018.